UNIVERSITI TEKNOLOGI MARA

COMPARATIVE ANALYSIS OF SPEECH DETECTION MODELS WITH A FOCUS ON THE JAPANESE LANGUAGE

MUHAMMAD ALIFF AIMAN BIN ZOLKIFELI

MSc

March 2025

UNIVERSITI TEKNOLOGI MARA

COMPARATIVE ANALYSIS OF SPEECH DETECTION MODELS WITH A FOCUS ON THE JAPANESE LANGUAGE

MUHAMMAD ALIFF AIMAN BIN ZOLKIFELI

Dissertation submitted in partial fullfillment
of the requirements for the degree of
Master of Computer Science

College of Computing, Informatics and Mathematics

March 2025

ABSTRACT

Automatic speech recognition for formal Japanese remained challenging due to script variation and sensitivity to front-end processing, and comparative evidence across model families was limited under matched conditions. This study aimed to determine which ASR model family performed best for formal Japanese transcription when preprocessing, feature extraction, and evaluation were standardized. A Japanese TEDx dataset with manual subtitles was collected, cleaned, resampled to 16 kHz mono, segmented using VAD, filtered to remove low-quality or non-speaker segments, and split into 80/10/10 train/validation/test sets. Three model families were trained and evaluated on the same splits. A Kaldi GMM–HMM pipeline (mono→tri3) with MFCC+CMVN, a SpeechBrain CRDNN–CTC system using log-Mel features with greedy and beam decoding, and fine-tuned Whisper variants from tiny to turbo. Performance was measured using CER as primary metrics, WER, and RTF. Fine-tuned Whisper achieved the best accuracy, reaching 4.28% CER (turbo) and 4.22% WER (medium), while CRDNN–CTC achieved the fastest decoding with 0.0129 RTF and 15.23% CER. The best Kaldi stage (tri3) achieved 19.48% CER with 0.251 RTF. These results provided an actionable accuracy–latency trade-off guide for selecting Japanese ASR pipelines for formal transcription.

Keywords: Japanese ASR, formal speech transcription, GMM–HMM, CRDNN–CTC, Whisper fine-tuning

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER ONE
RESULTS AND DISCUSSION

1.1 Introduction

In this chapter, the result of the workflow explained in Chapter 3 is presented. The result covered each steps from data collection, preprocessing, model training, fine-tuning, and finally the metrics result for all tested models is presented. The performance is measured using character error rate (CER), word error rate (WER), and real-time factor (RTF), following the research objective explained in Chapter 1 and the gap identified in Chapter 2. The analysis in this chapter focuses on two goals. The first goal is measure and compare the accuracy of the selected models, and second goal is to measure how fast the model can generate output based on the inputted audio (Ando & Fujihara, 2021).

1.2 Data Collection and Preprocessing Results

1.2.1 Data Source Selection Outcomes

This study gathered data from publicly available Japanese TEDx talks and only videos that contain manual subtitle was selected from the playlist. Videos with auto-generated subtitles or subtitles in languages other than Japanese will be excluded to reduce transcript noise.

Table 1.1 Video selection outcomes for TEDx Japanese data collection.

Item	Count
Playlist items queried	1574
Manual Japanese subtitles found	862
Auto-generated only / none	712
Download completed	862

Since the data source only utilizes data from manual transcription, it is align with the testing objective that is to produce a readable and standardized transcript while at the same time reducing the noise during training and evaluation. This is important because Japanese language can have words with different meaning with

same pronunciation. If this kind of data used as training data, it will resulting in model performance to become worsen.

(Koenecke et al., 2020).

1.2.2 Transcript Cleaning and Normalization Outcomes

After the subtitle is downloaded, the raw data from the subtitle was normalized into a reference transcript that acts as the source of truth. This cleaning process involves removing formatting tags, removing speaker labels, and discarding non-speech annotations such as `[laughter]`, `(applause)`. The output from this process is a clean transcript with a timestamp. Another benefit of removing the labels is to reduce noise as things like bracket events will increase the error rate because when calculating the error number, the bracket event will be treated as a deletion from the reference transcript.

```
Timestamp: 00:00:01.000 --> 00:00:03.200
RAW      : [applause] みなさん、こんにちは。
CLEANED  : みなさん、こんにちは。

Timestamp: 00:00:03.200 --> 00:00:06.000
RAW      : Speaker 1: 今日はAIについて話します。
CLEANED  : 今日はAIについて話します。

Timestamp: 00:00:06.000 --> 00:00:09.000
RAW      : (笑) それでは始めましょう。
CLEANED  : それでは始めましょう。

Timestamp: 00:00:09.000 --> 00:00:12.000
RAW      : <i>重要なのは</i> 継続です。
CLEANED  : 重要なのは 継続です。

Timestamp: 00:00:12.000 --> 00:00:16.000
RAW      : [music] (拍手) 本日はありがとうございます。
CLEANED  : 本日はありがとうございます。
```

Figure 1.1 Raw vs cleaned transcript after preprocesing

As shown in Figure 1.1, the raw subtitle contains non-speech items like formatting symbols and additional annotations that will be very useful for human viewing the video but since it is not part of the intended script, it will cause noise. Another thing to be taken into consideration is that the post-processing also needs to clean these kinds of annotations in the hypothesis transcript. This will improve the fair-

ness when comparing these models because the CER/WER is measured based on the speech content rather than subtitle formatting. (Ando & Fujihara, 2021)

1.2.3 Audio Standardization Outcomes

After the audio is downloaded, it will be converted into a consistent 16 kHz, mono, PCM16 format. The audio needs to be formatted to ensure compatibility when used in Kaldi, SpeechBrain, and Whisper training pipelines. By standardizing the sampling rate, it will reduce the differences caused by audio encoders and also 16 kHz, mono, PCM16 is the standard data expected from traditional pipelines (MFCC + CMVN) and modern neural pipelines (log-Mel filterbanks) (Ghoshal et al., 2011; McFee et al., 2025). This step also will support the "apples-to-apples" comparison goal stated in Chapter 1 by using the same data format across all models (Xu et al., 2023). Figure 1.2 below shows the difference of downloaded and resampled audio.

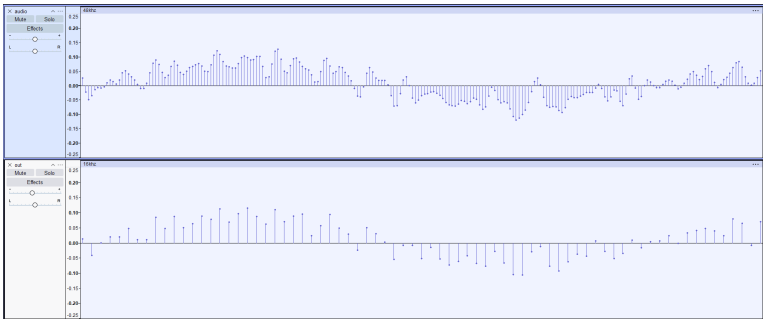

Figure 1.2 Example of audio standardization via resampling (44.1 kHz to 16 kHz).

1.2.4 VAD Segmentation Outcomes

The downloaded audio length is approximately 20 minutes long, which is not suitable for direct use as training data. By segmenting the long audio into shorter utterances, it will be more practical to use as training data. Additionally, models like Whisper, which is based on transformers, will cause memory issues because long sequences of training data will increase memory usage and decrease the stability of the models during training and fine-tuning. The segmentation process is done using a lightweight segmentation mechanism widely used in speech preprocessing pipelines named WebRTC VAD (Blum et al., 2021).

Table 1.2 VAD segmentation results.

Item	Count / Value	Notes
Total chunks produced	74802	After segmentation and slicing
Chunks removed (too short)	11881	$\leq$2 s
Avg chunk duration (s)	6.605	After filtering short clips
Max chunk duration (s)	20	Enforced by slicing

Table 1.2 above showed the summary of the data after segmentation. The average audio length became 6.6 seconds, which are closer to utterance-level speech than long-form audio. This will eliminate the memory usage problem mentioned earlier, as the longest audio is only 20 seconds. Another reason to chunk the audio is to prevent the RTF meafrom surement creating a spike because of limits when decoding the audio.

1.2.5 Non-Speaker / Low-Quality Audio Filtering Outcomes

The filtering process is carried out based on the requirement in Chapter 1, which states that the preprocessing method will heavily impact performance, especially for audio that contains too much noise, such as non-linguistic events or multi-speaker overlap (Latif et al., 2020). This type of data needs to be removed because it may contain a weak or wrong subtitle alignment even when the segment has manual subtitles.

Table 1.3 Audio filtering outcomes (diarization + CLAP).

Item	Count	Notes
Chunks before filtering	62921	Output from VAD
Removed by speaker-structure check	13758	Dominant speaker ratio threshold
Removed by clap/laughter threshold	5596	CLAP score thresholds
Chunks after filtering (final)	43567	Used to build manifests

The result in the table above shows that after removing non-speaker and low-quality audio, only 43567 chunks remain from 62921. This shows that it is a trade-off between the quantity and the quality of the audio, where filtering will reduce the number of dataset size but increase the segment with clean Japanese speech. The reliability of CER/WER is also increased because the test reference is less affected by non-speech artifacts and misalignment noise (Ando & Fujihara, 2021).

1.2.6 Transcript-to-Audio Mapping and Manifest Outcomes

After the audio is segmented and cleaned, each chunk will be paired with the reference transcript from the cleaned subtitle earlier. From there, the manifest containing the audio and transcript data is created. The objective of creating the manifest is to improve reproducibility when splitting the data into train and test splits. This will also ensure that the differences in the end result are only due to the model itself and not on inconsistent data splits (Ravanelli et al., 2021).

	ID	path	duration	transcript
0	1367_Why_a_city_...	/content/data/wa...	2.10	なぜそこの女川町を選んだのか。
1	398_Career_educa...	/content/data/wa...	3.69	あんなに堂々と 生き生きと発表す...
2	1250_１_TEDxWakay...	/content/data/wa...	3.81	こういう救出はこの到着部隊から考...
3	282_The_Regions_...	/content/data/wa...	10.89	これ人工衛星の話じゃないですか。...
4	1336_Bringing_a_...	/content/data/wa...	4.92	庭園 建物 様々なものづくりを発...
5	1348_A_face_to_f...	/content/data/wa...	3.90	たくさんの人を山に案内する中で ...
6	611_TEDxUSH_chun...	/content/data/wa...	3.99	何か自分の変わる人生が 変わるき...
7	1348_A_face_to_f...	/content/data/wa...	5.13	森は仕事として関わる一部の人間に...
8	1269_TEDxHaneda_...	/content/data/wa...	11.82	でも これって誰が悪いとか そう...
9	658_The_challeng...	/content/data/wa...	14.28	ハイビスカスをつけて 踊りながら...

Figure 1.3 Data after mapping audio and transcript into manifest format

Figure above shows some of the data from the manifest data. The manifest data contains four column that is ID, path, duration, and transcript. The ID data is used to differentiate each chunk from the main audio file name. Duration and path are the data of the audio file, keeping records of where the audio is stored and the duration of the audio file. Lastly, the transcript is the reference transcript that will be used as labelled data when training models and as a reference transcript when testing the models.

1.2.7 Train/Validation/Test Split Results

The final dataset was split into 80% training, 10% validation, and 10% testing. Table 1.4 reported split sizes.

Table 1.4 Dataset statistics and train/validation/test split summary.

Split	#Utterances	Duration (hrs)	Average (s)
Train	32852	63.3	6.941
Valid	4357	8.3	6.861
Test	4358	8.1	6.731

The split statistics show comparable average durations across train/valid/test, which reduces the risk that one split systematically contains longer or harder utterances. Keeping the test split fully held out supports a cleaner interpretation of generalization and makes the evaluation consistent with comparative reporting in prior Japanese ASR studies (Ando & Fujihara, 2021; Karita et al., 2021).

1.3 Model Training and Fine-Tuning Results

1.3.1 Kaldi GMM–HMM Training Outcomes

Kaldi training consisted of four stages of training, which are mono, tri1, tri2, and tri3. The later stages were trained based on the previous stages' aligned data. This convention follows the Kaldi recipe logic, where increasing the context-dependent model will improve the alignment and feature-space transform. The consistent improvement aligns with established reports with other training recipes, such as CSJ, where later triphone and SAT will produce better accuracy gain (Ghoshal et al., 2011; Trmal, 2015)

Table 1.5 Kaldi GMM–HMM training dynamics based on objective function improvement per frame. Peak improvements occur early and decrease toward near-zero values, indicating convergence.

Stage	Peak impr./frame	Peak iter	Final impr./frame	Stable iter (< 0.02)
mono	0.4289	1	0.0110	32
tri1	0.2022	3	0.0026	30
tri2	0.4820	1	0.0034	30
tri3	0.2975	3	0.0034	30

The trend line in Table 1.5 above shows that most of the huge parameter updates occurred early in the training, then followed by small improvements until it hit a plateau and convergence was approached. This pattern is expected for GMM–HMM training and aligns with typical Kaldi diagnostics used to detect training stability and saturation (Ghoshal et al., 2011).

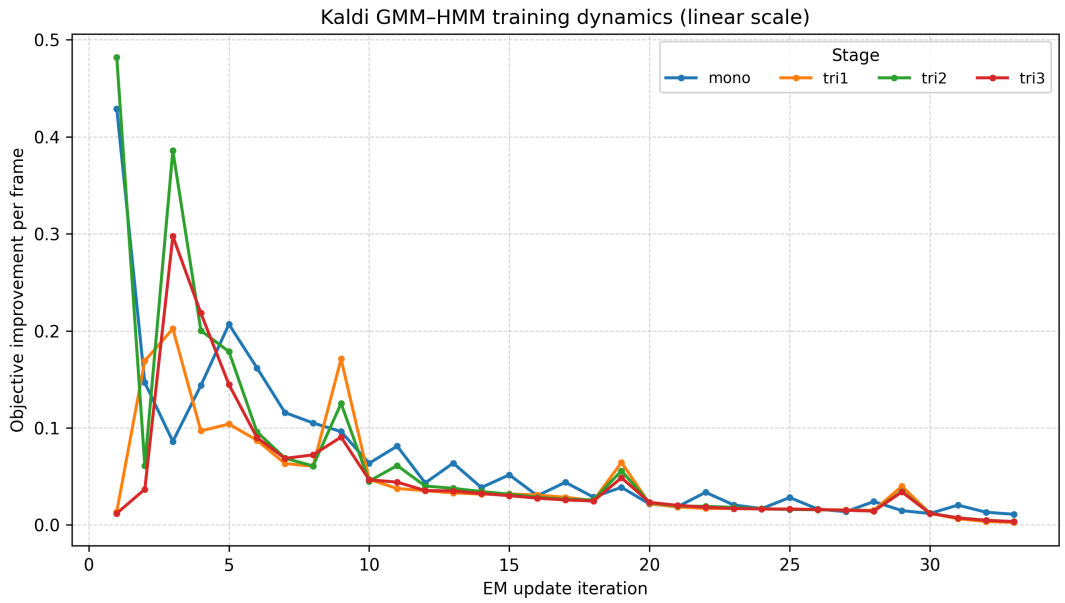

Figure 1.4 Kaldi GMM-HMM training dynamics in linear scale

1.3.2 CRDNN–CTC Training Outcomes

The CRDNN-CTC model was trained using the manifest prepared in the pre-processing phase and a character vocabulary derived from the training script. The training was done over 70 epochs, where the starting training loss was 5.44 at the start of the training and became 0.0775 at the end, with the lowest CER at 0.2272 during epoch 68. After that, the best checkpoint was selected using validation tracking and later evaluated on the test split (reported in Table 1.10). Table 1.6 summarized the epoch-by-epoch training dynamics from `data.txt`.

For the decoding part using CTC, it is well aligned with Japanese transcription due to its ability to use monotonic alignment without needing a pronunciation dictionary. The reference transcript also does not need to be tokenized into words, which is very useful for languages that do not have space between words in a sentence (Watanabe et al., 2018).

Table 1.6 CRDNN–CTC training metrics across epochs.

Epoch	Train loss	Valid loss	CER
1	5.4400	5.2833	0.9983
2	4.0100	3.0769	0.6428
5	1.9100	1.6922	0.4555
10	1.1200	1.3025	0.3372
15	0.7760	1.0662	0.2906
20	0.5770	1.0610	0.2783
25	0.3130	1.0613	0.2629
30	0.2250	1.0297	0.2483
35	0.1740	1.0951	0.2404
40	0.1360	1.1158	0.2363
50	0.2050	1.0767	0.2437
60	0.1090	1.1674	0.2352
70	0.0775	1.2433	0.2309

Training curves below show that learning occurs very strongly in the early stages and then followed by small changes later in the training as the epochs increase. The error rate value starting from a very high number that is 0.9983 which shows that the prediction is basically random at this point of training. As the train loss and valid loss decrease, the CER also decreased. The figure below shows that training and validation loss as the epoch increases, with valid loss becoming flat first, followed by training loss. The figure also shows that CER is decreasing aggressively at the start of the training and then becomes stable in the middle and maintains the trend until the end of the training.

Figure 1.5 Training vs validation loss

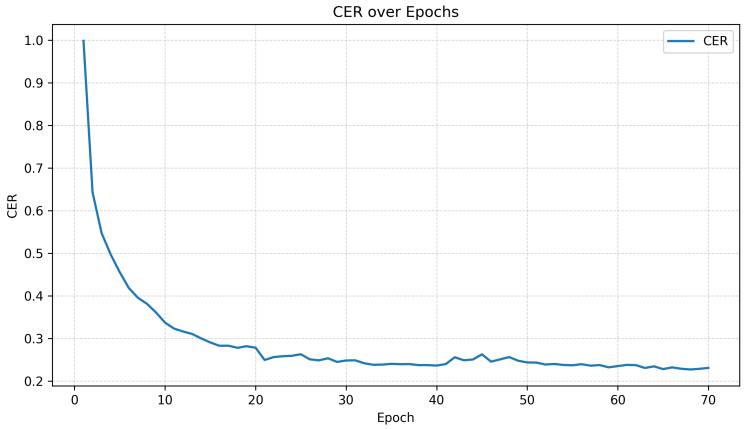

Figure 1.6 CER over Epochs

1.3.3 Whisper Fine-Tuning Outcomes

The Whisper model fine-tuning is using a pre-trained model provided by OpenAI, where the model is built upon a large-scale weak supervision, which has been shown to improve robustness under domain mismatch and reduce the reliance on task-specific feature engineering (Radford et al., 2023). Previous studies from Bajo et al. (2024) show that a notable improvement is it achieved when fine-tuning the base multi-language models by adapting to the Japanese language. The fine-tune stability can be seen from the loss vs epoch values below.

Table 1.7 Whisper fine-tuning loss across epochs (lower is better).

Epoch	tiny	base	small	medium	turbo
1	0.6626	0.3683	0.1988	0.0747	0.2137
2	0.3636	0.2106	0.0820	0.0415	0.1361
3	0.2862	0.1466	0.0413	0.0236	0.0911
4	0.2327	0.1034	0.0216	0.0130	0.0591
5	0.1930	0.0731	0.0115	0.0071	0.0362
6	0.1636	0.0520	0.0061	0.0035	0.0198
7	0.1426	0.0380	0.0033	0.0016	0.0095
8	0.1289	0.0296	0.0016	0.0012	0.0041
9	0.1205	0.0252	0.0008	0.0006	0.0016
10	0.1173	0.0234	0.0006	0.0003	0.0003

The larger model, like medium and large-turbo, becomes stable faster at only epoch 5, while the smaller model, like tiny, becomes stable become stable after epochs 9. Also worth noting that the loss value after epochs 10 is different eventhough the

training become stable showing that the larger model is more quick to adapt to the training data compared to the smaller model.

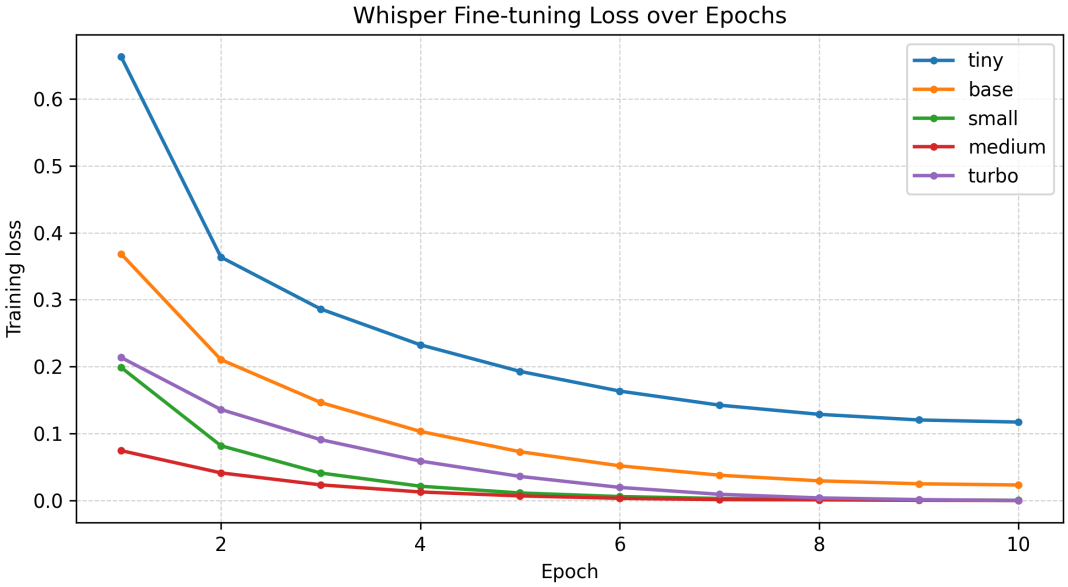

Figure 1.7 Whisper Fine-tuning loss over Epochs

1.4 Evaluation Results (CER, WER, and RTF)

1.4.1 Overall Results and Discussion

Table 1.8 above shows that Whisper achieved the highest accuracy with the lowest CER and WER overall, followed by CRDNN–CTC which is second in accuracy but has the highest decoding speed. And then, the traditional model Kaldi shows a good result and strong gains from staged training but remains less accurate compared to the neural approaches.

Based on the literature review in Chapter 2, the pattern is consistent with past studies where newer end-to-end models that build using transformers perform better compared to the older pipeline in terms of accuracy. However, architectural and decoding choices have a strong impact on latency (Radford et al., 2023; Xu et al., 2023).

Table 1.8 Overall comparison of ASR systems on the test split (lower is better for CER/WER/RTF).

Model	CER (%)	WER (%)	RTF
GMMHMM-mono	49.49	53.43	0.6500
GMMHMM-tri1	24.95	29.31	0.3410
GMMHMM-tri2	21.30	25.59	0.2480
GMMHMM-tri3	19.48	23.57	0.2510
CRDNNCTC-1	15.23	22.87	**0.0129**
CRDNNCTC-2 (beam)	15.12	22.70	0.0147
Whisper-tiny	10.72	11.61	0.0297
Whisper-base	5.67	5.89	0.0413
Whisper-small	4.34	4.30	0.0737
Whisper-medium	4.29	**4.22**	0.1605
Whisper-turbo	**4.28**	4.23	0.0959

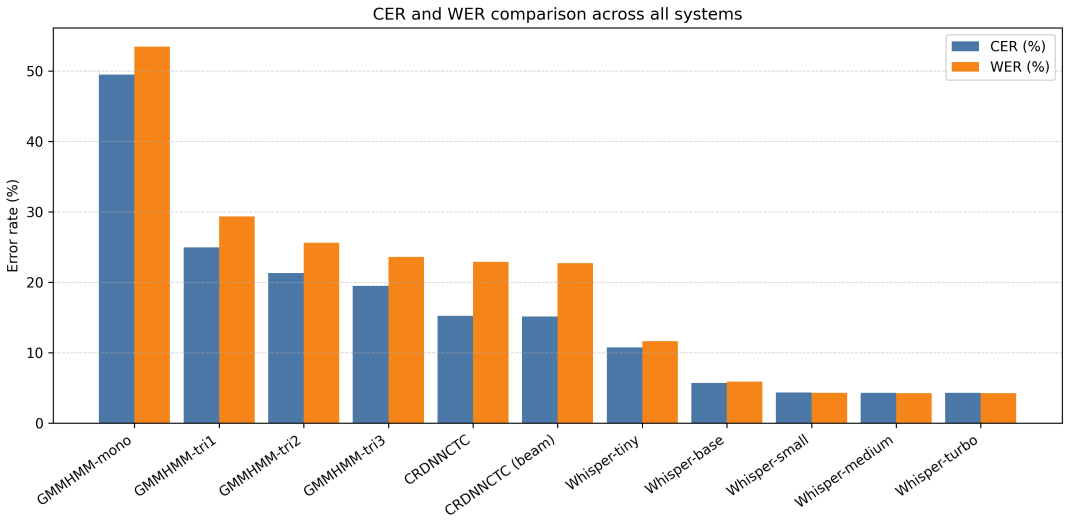

Figure 1.8 CER and WER comparison across all system

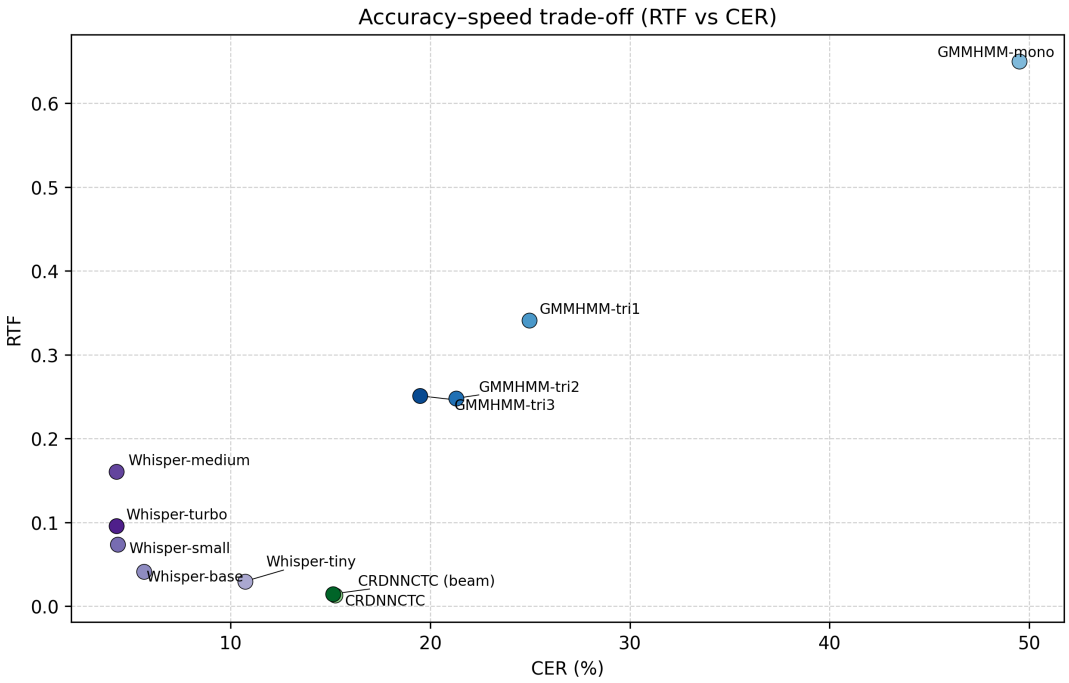

Figure 1.9 Accuracy speed trade-off

The ranking for accuracy is consistent across both CER and WER, where Whisper variants are the best in terms of precision while placing CRDNN–CTC a little bit behind in terms of accuracy, which sits in the middle, and Kaldi is the weakest, even at tri3. In contrast, from the RTF performance, CRDNN–CTC runs the fastest, with Whisper slowing down as model size increases.

1.4.1.1 Why the overall ranking occurs

The observed ranking (Whisper best accuracy, CRDNN–CTC fastest, Kaldi behind) is largely explained by differences in **(i) prior knowledge**, **(ii) decoding mechanism**, and **(iii) dependence on linguistic resources**. First, Whisper benefits from large-scale pretraining on diverse speech and text patterns, which helps it resolve ambiguous Japanese readings (homophones) and maintain grammatical consistency even when acoustics are imperfect (Radford et al., 2023; Xu et al., 2023). Second, CRDNN–CTC decoding is **non-autoregressive**: it predicts frame-level posteriors and collapses them into text with greedy/beam search, which is computationally lightweight, yielding very low RTF (Chen et al., 2020; Watanabe et al., 2018). Third, Kaldi's GMM–HMM pipeline depends more heavily on feature engineering, lexicon/language modeling choices, and alignment quality; staged training improves

it substantially, but its modeling capacity is still limited compared to modern end-to-end systems under domain variability (Ando & Fujihara, 2021; Ghoshal et al., 2011).

Overall finding that can be concluded is that Whisper models handled Japanese TEDx talks better than others. This can be seen from the CER and WER in speech recognition tasks. However, the real-time performance aspect was something else that was unexpected, as the CRDNN–CTC system processed audio faster than amaking it ll, suitable for real-time usage. Training helped GMM-HMM a lot at each stage. However, when measured against deep learning systems, its performance stayed behind.

The overall outcomes align with the objective explained in Chapter 1 and with past studies in reviewed in Chapter 2. ASR systems for Japanese speech recognition perform better when they handle symbol patterns and draw on broad pre-trained knowledge. Meanwhile, older processing methods still offer clearer logic paths and organized learning setups. However, their precision tends to drop when used across changing environments (Ando & Fujihara, 2021; Koenecke et al., 2020; Xu et al., 2023).

1.4.2 Kaldi GMM–HMM Results and Discussion

Starting at mono and moving through tri3, the performance of GMM-HMM steadily increases. As the model advances, CER drops and falls from nearly 49.49% to 19.48%, a drop of more than 60%. WER follows a similar trend, first decreasing from over 53.43% to 23.57%, cutting errors by roughly half. Most of the progress happens early, particularly between mono and tri1. After that point, each step only lowers the error rate by a small amount but continuously.

Table 1.9 Kaldi GMM–HMM results across training stages on the test split.

Stage	CER (%)	WER (%)	RTF
mono	49.49	53.43	0.6500
tri1	24.95	29.31	0.3410
tri2	21.30	25.59	0.2480
tri3	19.48	23.57	0.2510

Smaller improvements from tri1 to tri3 show an expected behavior of conventional pipelines, where shifting to context-sensitive triphones results in notable

progress. Additionally, subsequent enhancements via feature adjustments and SAT add modest improvements (Ghoshal et al., 2011; Trmal, 2015). In terms of real-time usability, GMM-HMM shows that the model's performance in terms of RTF becomes better as efficiency holds despite deeper layers in GMM-HMM models.

1.4.2.1 Why Kaldi improves sharply early and then saturates

The large jump from mono to tri1 is expected because monophone models treat each phonetic unit independently, while triphones model **context-dependent** realizations, which is critical when coarticulation changes acoustic realizations rapidly in continuous Japanese speech (Ghoshal et al., 2011). Once triphones are introduced, many systematic substitution/deletion errors caused by context mismatch are reduced, explaining the steep initial drop in CER/WER.

From tri1 to tri3, gains become smaller because improvements increasingly come from **refining the same underlying representation** rather than adding fundamentally new modeling power. LDA+MLLT (tri2) improves feature-space discrimination and can tighten class separation, while SAT (tri3) reduces speaker variability by adapting transforms, but these steps cannot fully overcome limitations of GMM acoustic modeling and the dependency on pronunciation/LM coverage. As a result, later stages show diminishing returns, consistent with typical Kaldi diagnostics and prior recipe behavior (Ghoshal et al., 2011; Trmal, 2015).

The relatively stable RTF across tri2/tri3 suggests that decoding complexity is dominated by the search graph and pruning settings, while better acoustic scores mainly improve accuracy rather than drastically changing runtime.

The increase in accuracy from mono to tri1 model is because the triphone model uses context-sensitive units, compared to mono models that use fixed units.The next gain is from training the models with tri2 and tri3, which results in a slightly increased performance. The increased gain in tri2 is due to better signal processing via LDA and MLLT. Speaker-specific adjustments during training also help to improve the models by implementing SAT in tri3. However, each subsequent step added less gain than the one before it, as limits were neared within the setup and model capabilities.

One key point from these findings is that the way to increase the accuracy

of traditional models is by using step-by-step training. To further improve this kind of model, better lexicons, refined language modeling, or discriminative methods are required. The results shown from these models align with earlier studies where traditional Japanese ASR systems gain significantly from careful tuning of stages. However, the final accuracy score is still behind compared to advanced end-to-end models. Especially under difficult conditions, where newer Transformer-based models tend to perform better, a similar pattern also appears in studies conducted by (Ando & Fujihara, 2021; Trmal, 2015).

1.4.3 CRDNN–CTC Results and Discussion

In the CRDNN-CTC results, the error rate decreased slightly when using beam decoding compared to greedy decoding. The decrease from 22.87% to 22.70% resulted in a 0.17% difference between both decoders. The decoding speed increased significantly by about 13.95% from greedy decoding to beam decoding. However, both versions of CRDNN-CTC achieved the fastest decoding time compared to other models.

Table 1.10 CRDNN–CTC results on the test split (greedy vs beam).

Decoder	**CER (%)**	**WER (%)**	**RTF**
Greedy (CRDNNCTC-1)	15.23	22.87	**0.0129**
Beam (CRDNNCTC-2)	15.12	22.70	0.0147

A slight difference in accuracy between greedy and beam decoding shows that the acoustic model has a greater influence, while beam search adds little gain. In this case, the performance leans more on the encoder's strength rather than advanced decoding methods. Prior work supports this pattern, which shows that greedy approaches often match beam results when language modeling plays a minor role (Chen et al., 2020). What stands out is how fast both setups operate, delivering very low RTF values. Their speed makes them practical choices for real-time applications where delay matters.

1.4.3.1 Why beam search gives only a small gain

The small WER/CER improvement from greedy to beam decoding indicates that most remaining errors are **acoustic/encoder-limited** rather than resolvable by

stronger sequence constraints. In other words, if the model's posterior distribution is already sharply peaked, beam search explores alternatives but finds few better candidates, producing only marginal gains while adding computation (Chen et al., 2020). This is also consistent with character-based Japanese decoding where a lightweight LM may not strongly disambiguate particles or short function words unless the acoustic evidence is already close.

1.4.3.2 Why CRDNN–CTC achieves the best RTF

CRDNN–CTC is fast because CTC decoding does not require step-by-step autoregressive generation. Once frame-level posteriors are computed, decoding is mainly a collapse operation (plus optional beam expansion), avoiding the repeated decoder forward passes that sequence-to-sequence models require for every output token. This structural difference directly explains why CRDNN–CTC reaches the lowest RTF values among all tested systems (Watanabe et al., 2018).

In CRDNN-CTC models, the differences in decoding between greedy and beam search were only slight. This is because most performance was gained from the acoustic model, and the chosen beam settings provided limited additional benefit relative to the added decoding cost. Across the tests, each CRDNN version ran faster than Whisper and Kaldi when measured using RTF. However, while beam tuning helped slightly, its extra computation did not justify the effort needed.

This also shows that CRDNN–CTC performs effectively when quick response times are crucial, like in live subtitles or spoken dialogue systems. This is because, CRDNN model outputs frame-level probabilities, then decoding is either greedy or a lightweight beam search over characters/subwords. No need to run a separate decoder network step-by-step for every output token which make it faster then Whisper and traditional models. Earlier studies on CTC-driven Japanese speech recognition support this outcome, demonstrating strong character error rates despite minimal context or continuous input modes (Chen et al., 2020).

1.4.4 Whisper Fine-Tuning Results and Discussion

Transcription accuracy reached its highest with Whisper fine-tuning. Moving up from tiny to base brought a clear improvement, where the WER dropped from

11.61% down to 5.89%. A further step to small model reduced the WER again, landing at 4.30%. Progress beyond that slowed, where medium only edged ahead by 1.86%, lowering WER to 4.22%. At the same time, processing lagged behind, and RTF climbed past 0.16. Yet, Whisper-turbo matched medium closely, hitting 4.23% WER while responding quicker than medium, with an RTF under 0.1.

Table 1.11 Whisper fine-tuning results by model size on the test split.

Variant	**CER (%)**	**WER (%)**	**RTF**
tiny	10.72	11.61	0.0297
base	5.67	5.89	0.0413
small	4.34	4.30	0.0737
medium	4.29	**4.22**	0.1605
turbo	**4.28**	4.23	0.0959

Results reveal how speed ties to accuracy where bigger models gain precision only until gains slow, demanding far more computing power. Earlier work supports this because pre-trained Transformers handle Japanese speech well, though size must suit real-world limits (Bajo et al., 2024; Radford et al., 2023). Here, Whisper-turbo strikes a useful middle ground, holding near top-tier word error rates while cutting response time compared to Whisper-medium.

1.4.4.1 Why Whisper gains saturate with larger sizes

The steep improvement from tiny $\rightarrow$ base $\rightarrow$ small reflects increased model capacity to exploit long-range context and stronger internal language modeling, which is particularly helpful for Japanese where short particles and homophones require contextual resolution (Radford et al., 2023). However, the small improvement from small to medium suggests a **ceiling effect** under this dataset and evaluation setup: remaining errors may be dominated by subtitle-reference mismatch, residual normalization differences (numbers/punctuation), or intrinsically ambiguous segments, limiting how much extra capacity can help.

1.4.4.2 Why Whisper-turbo is competitive

Whisper-turbo achieves near-medium accuracy with substantially lower RTF, suggesting it offers a better accuracy–latency balance for practical deployment. Given

the small accuracy gap between medium and turbo (only 0.01 WER difference in Table 1.11) but a much lower RTF, turbo is a strong candidate when real-time or near-real-time constraints matter, while medium remains best when maximum accuracy is prioritized.

From the results produced by the Whisper models, a conclusion can be made that bigger models boosted Whisper's performance, yet slowed things down noticeably. The medium models version achieved the lowest WER, and the lowest CER is achieved by turbo models. However, when comparing these two models in terms of RTF, turbo models have an advantage compared to medium. The RTF becomes higher as the model size is larger because larger models need more resources to process the tokens and output the results when decoding.

The result is in line with the past study by Radford et al. (2023) where using a structured environment and a dataset like TEDx event, Whisper was able to achieve good results because of broad pre-trained systems able to manage sound variation efficiently, especially once adapted through targeted training data. However, one thing needs to be considered when choosing Whisper for real applications is that the resource usage is higher compared to other models.

1.5 Comparative Assessment

This section consolidates the evaluation results into a single comparative view, focusing on (i) accuracy, (ii) decoding speed, and (iii) sensitivity to evaluation choices for Japanese.

1.5.1 Accuracy comparison (CER/WER)

Across all systems, Whisper fine-tuned variants achieved the best transcription accuracy, with WER around 4.22–4.23% and CER around 4.28–4.29% (Table 1.8). CRDNN–CTC achieved mid-range accuracy (WER $\approx$ 22.70–22.87%), while Kaldi tri3 remained behind (WER 23.57%), despite large gains from staged training. This pattern suggests that pretrained encoder–decoder Transformers generalize better to TEDx speech and handle linguistic ambiguity more effectively than both conventional pipelines and smaller end-to-end encoders under the same data conditions (Radford

et al., 2023; Xu et al., 2023).

1.5.2 Speed comparison (RTF) and deployment implications

CRDNN–CTC achieved the lowest RTF (0.0129–0.0147), making it the most suitable candidate for real-time scenarios such as live captions or interactive systems. Whisper models showed a clear latency increase with size, from tiny (0.0297) to medium (0.1605). Kaldi decoding remained real-time capable (RTF $\approx$ 0.25 for tri2/tri3) but did not match the neural systems in accuracy. Overall, these results reveal a practical trade-off: if strict latency is required, CRDNN–CTC is preferred; if highest accuracy is required, Whisper is preferred; and Kaldi remains valuable when interpretability and traditional resource control (lexicon/LM) are priorities (Ando & Fujihara, 2021; Ghoshal et al., 2011).

1.5.3 Accuracy–speed trade-off

The combined plots (CER/WER vs RTF) indicate three distinct operating regions. (1) **High accuracy / moderate speed:** Whisper-medium and Whisper-turbo provide the strongest recognition quality but require more computation. (2) **Moderate accuracy / very fast:** CRDNN–CTC provides the best runtime performance with acceptable accuracy for time-critical use. (3) **Lower accuracy / real-time:** Kaldi tri3 remains usable in real-time but is less competitive for TEDx Japanese accuracy.

1.5.4 Sensitivity to Japanese WER tokenization

The tokenization analysis (Table 1.16) shows that Japanese WER values can change drastically depending on whether word boundaries are inserted. Without tokenization, WER is not comparable across systems and can exaggerate differences due to the "single-token sentence" effect. Therefore, tokenized WER is the meaningful metric for cross-model comparison in this study, and the tokenization tool/settings should be treated as part of the evaluation protocol (Ando & Fujihara, 2021).

1.5.5 Overall recommendation based on study objectives

If the primary objective is **maximum transcription quality** for archival or reporting, Whisper-medium/turbo is recommended. If the primary objective is **low-**

latency decoding for live usage, CRDNN–CTC is recommended. Kaldi remains useful as a strong baseline and for scenarios where traditional pipeline control (lexicon/LM tuning and staged diagnostics) is desirable, but it is less competitive in final accuracy under this dataset.

1.6 Error Analysis

Across the qualitative examples (Tables 1.12–1.15), three recurring error patterns were observed. First, function-word deletions and minor particles were common in Kaldi and CRDNN outputs (e.g., missing small grammatical markers), which can occur under fast speech or reduced articulation and contribute disproportionately to CER when characters are missing. Second, named entity and word-choice substitutions appeared in the lower-performing systems (e.g., shorthand forms or near-homophones), reflecting limitations in lexical modeling and context resolution that are known challenges in Japanese due to ambiguous readings and context sensitivity (Curtin, 2020; Ito et al., 2016, 2017). Third, formatting and normalization inconsistenciess (e.g., number rendering or punctuation/spacing variation) were visible across systems, which motivates the post-processing and text formalization emphasis discussed in Chapter 1 (Ando & Fujihara, 2021).

Although Whisper produced the fewest errors overall, the examples show it still made small mistakes such as minor polite-form variation or subtle substitutions. This supports the view that even strong pretrained models may require additional normalization rules or domain-specific constraints to achieve fully standardized outputs for archival or reporting use cases (Radford et al., 2023).

Table 1.12 Qualitative transcription example 1: Wording errors.

System	Text / diff vs reference
Reference	来週、東京大学でAIの安全性について講演します。
GMM-HMM	来週[、→␣]東京大[-学-]でAIの安全[-性-]について[講→公]演します
CRDNN–CTC	来週、東京大学でAIの安全[-性-]について講演します
Whisper - turbo	来週、東京大学でAIの安全性について講演します

Table 1.13 Qualitative transcription example 2: Polite-form variation.

System	Text / diff vs reference
Reference	その結果をすぐに共有して、次の手順を決めましょう。
GMM-HMM	その結果[を→␣]すぐ[-に-]共有して[、→␣]次の手順[を→␣]決めましょう
CRDNN–CTC	その結果をすぐ[-に-]共有して、次の手順を決め[ま→よ][-し-][-ょ-]う
Whisper - turbo	その結果をすぐに共有して、次の手順を決め[ま→よ][-し-][-ょ-]う

Table 1.14 Qualitative transcription example 3: Paraphrase.

System	Text / diff vs reference
Reference	要するに、私たちは失敗から学ぶ必要があります。
GMM-HMM	要する[に、→␣]私たちは失敗から学ぶ[必→␣][要→ひ][+つ+][+よ+][+う+]があります
CRDNN–CTC	[要→つ][す→ま][る→り][-に-]、私たちは失敗から学ぶ必要があります
Whisper - turbo	要するに、私たちは失敗から学ぶ必要があ[り→る][-ま-][-す-]

Table 1.15 Qualitative transcription example 4: Numbering format.

System	Text / diff vs reference
Reference	2024年にはクラウドへの移行が加速しました。
GMM-HMM	[2→二][0→千][2→二][4→十][+四+]年にはクラ[ウ→ン]ドへの移行が加速しました
CRDNN–CTC	2024年にはクラウドへの移行が加速しました
Whisper - turbo	2024年にはクラウドへの移行が加速しました

1.7 Impact of Preprocessing and Postprocessing Choices on Results

Because all systems used the same cleaned and standardized dataset, differences in accuracy and speed mainly reflected model architecture and decoding strategy rather than evaluation handling. Nevertheless, the preprocessing steps in Chapter 3 (manual subtitle selection, cleaning, VAD chunking, and filtering) were designed to reduce transcript noise and non-speech audio, which supported stable training and fair comparison across model families (Ando & Fujihara, 2021; Latif et al., 2020).

A issue thet need to be addressed during postprocessing is that when decoding for wer, it resulting in very high WER because japanese is consider as one word for each sentence since it dows not have spaces. So the strategy is to use tokenization to tokenize the sentence into words first using python library first before calculate the WER.

Table 1.16 Best-performing model per family under each WER setting (values in %).

Model family	Best model	no tokenization	tokenized
GMM–HMM	tri3	81.71	23.57
CRDNN–CTC	beam	77.74	22.70
Whisper	medium	6.82	4.22

Table 1.16 highlights the best model from each family under two WER settings. A clear pattern is that **WER without tokenization is severely inflated for Japanese** in the GMM–HMM and CRDNN–CTC systems (e.g., 81.71% and 77.74%), while tokenized WER becomes much lower and more interpretable (23.57% and 22.70%). This happens because standard WER assumes *space-delimited words*. When Japanese sentences are evaluated without inserting word boundaries, an entire sentence may be treated as *one token*. In that case, even a small mismatch (one missing particle, one substituted kanji, or punctuation differences) can cause the whole sentence-token to be counted as a substitution, producing an unrealistically high WER. After tokenization, the same errors are distributed across multiple word tokens, so the score better reflects *how much of the sentence was correct* rather than penalizing the sentence as a single unit.

The smaller gap observed for Whisper (6.82% $\rightarrow$ 4.22%) suggests that Whisper outputs are already closer to the reference in terms of overall lexical choice and normalization, so boundary decisions affect WER less. In contrast, GMM–HMM and CRDNN–CTC still produce more local errors (particles, short deletions, minor substitutions) that may interact strongly with the tokenizer's boundary decisions and therefore change word-level scoring more dramatically. This finding supports the Chapter 1 motivation that **Japanese WER must be reported with clearly documented tokenization** for fair comparison across studies and systems (Ando & Fujihara, 2021).

Therefore, in this study, **tokenized WER is treated as the primary WER metric**, while no-tokenization WER is reported mainly to show how evaluation choices can distort conclusions for Japanese.

1.8 Limitations

Key limitations include TEDx-only speaking style, possible subtitle-to-speech mismatch, sensitivity of Japanese WER to tokenization, and limited ablation studies such as VAD thresholds, filtering thresholds, or language model variants.

The TEDx domain emphasizes relatively clear monologue speech, and therefore may not represent conversational Japanese or noisy real-world environments. In addition, subtitle references can still contain minor mismatches even when manually authored, which can place a ceiling on achievable CER/WER. Finally, the tokenization choice for Japanese WER can influence reported values, so consistent reporting and tool documentation are required for fair interpretation across studies.

1.9 Summary

This chapter presented results for each stage of the Chapter 3 pipeline and compared ASR performance across classical and neural approaches. Kaldi improved substantially across staged training, CRDNN–CTC provided the fastest decoding, and fine-tuned Whisper variants achieved the best transcription accuracy with clear speed trade-offs by model size. The next chapter concludes the study and proposes future improvements.

REFERENCES

Ando, S., & Fujihara, H. (2021). Construction of a large-scale japanese asr corpus on tv recordings. *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 6948–6952. https://doi.org/10.1109/ICASSP39728.2021.9413425

Bajo, M., Fukukawa, H., Morita, R., & Ogasawara, Y. (2024). Efficient adaptation of multilingual models for japanese asr. *arXiv preprint arXiv:2412.10705*.

Blum, N., Lachapelle, S., & Alvestrand, H. (2021). Webrtc. *Communications of the ACM*, *64*(8), 50–54. https://doi.org/https://doi.org/10.1145/3453182

Chen, J., Nishimura, R., & Kitaoka, N. (2020). End-to-end recognition of streaming japanese speech using ctc and local attention. *APSIPA Transactions on Signal and Information Processing*, *9*, e25. https://doi.org/10.1017/ATSIP.2020.23

Curtin, K. (2020). Japanese kanji power: A workbook for mastering japanese characters.

Ghoshal, A., Boulianne, G., Burget, L., Glembek, O., Goel, N., Hannemann, M., Motlíček, P., Qian, Y., Schwarz, P., Silovský, J., Stemmer, G., & Vesel, K. (2011). The kaldi speech recognition toolkit. *IEEE 2011 Workshop on Automatic Speech Recognition and Understanding*.

Ito, H., Hagiwara, A., Ichiki, M., Mishima, T., Sato, S., & Kobayashi, A. (2016). End-to-end neural network modeling for japanese speech recognition. *Journal of the Acoustical Society of America*, *140*, 3116–3116. https://doi.org/10.1121/1.4969755

Ito, H., Hagiwara, A., Ichiki, M., Mishima, T., Sato, S., & Kobayashi, A. (2017). End-to-end speech recognition for languages with ideographic characters. *2017 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 1228–1232. https://doi.org/10.1109/APSIPA.2017.8282226

Karita, S., Kubo, Y., Bacchiani, M. A. U., & Jones, L. (2021). A comparative study on neural architectures and training methods for japanese speech recognition. *Proceedings of the Annual Conference of the International Speech Communi-*

cation Association, INTERSPEECH, *2*, 2092–2096. https://doi.org/10.21437/INTERSPEECH.2021-775

Koenecke, A., Nam, A., Lake, E., Nudell, J., Quartey, M., Mengesha, Z., Toups, C., Rickford, J. R., Jurafsky, D., & Goel, S. (2020). Racial disparities in automated speech recognition. *Proceedings of the National Academy of Sciences of the United States of America*, *117*, 7684–7689. https://doi.org/10.1073/PNAS.1915768117/SUPPL_FILE/PNAS.1915768117.SAPP.PDF

Latif, S., Qadir, J., Qayyum, A., Usama, M., & Younis, S. (2020). Speech technology for healthcare: Opportunities, challenges, and state of the art. *IEEE Reviews in Biomedical Engineering*, *14*, 342–356.

McFee, B., McVicar, M., & and, D. F. (2025, March). *Librosa* (Version 0.11.0). Zenodo. https://doi.org/10.5281/zenodo.15006942

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *International conference on machine learning*, 28492–28518.

Ravanelli, M., Parcollet, T., Plantinga, P., Rouhe, A., Cornell, S., Lugosch, L., Subakan, C., Dawalatabad, N., Heba, A., Zhong, J., Chou, J.-C., Yeh, S.-L., Fu, S.-W., Liao, C.-F., Rastorgueva, E., Grondin, F., Aris, W., Na, H., Gao, Y., … Bengio, Y. (2021). Speechbrain: A general-purpose speech toolkit. https://arxiv.org/abs/2106.04624

Trmal, J. (2015). Kaldi CSJ recipe results (egs/csj/s5/results) [Accessed: 2025-12-29].

Watanabe, S., Hori, T., Karita, S., Hayashi, T., Nishitoba, J., Unno, Y., Yalta Soplin, N. E., Heymann, J., Wiesner, M., Chen, N., Renduchintala, A., & Ochiai, T. (2018). Espnet: End-to-end speech processing toolkit. *Interspeech 2018*, 2207–2211. https://doi.org/10.21437/Interspeech.2018-1456

Xu, C., Ye, R., Dong, Q., Zhao, C., Ko, T., Wang, M., Xiao, T., & Zhu, J. (2023). Recent advances in direct speech-to-text translation. *arXiv preprint arXiv:2306.11646*.